

## PROJECT REPORT

# Student Marks Analyser

*A menu-driven Python application for report cards, performance analysis and ranking*

|                           |                       |
|---------------------------|-----------------------|
| <b>Submitted by</b>       | Pranav Nath           |
| <b>Registration No.</b>   | 26BCE11041            |
| <b>Course</b>             | B.Tech CSE (Core)     |
| <b>Submitted to</b>       | MK Jayanthi           |
| <b>Institution</b>        | VIT Bhopal University |
| <b>Date of submission</b> | 29 September 2026     |

# Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

## 2. Introduction

At the end of every exam, a class teacher has to add up marks, work out percentages and grades, decide who passed, find the topper and prepare a rank list. When this is done by hand for a full class it takes a lot of time and a single wrong addition can change a student's rank.

The **Student Marks Analyser** is a Python program that does all of this from a simple terminal menu. The teacher enters (or randomly generates) marks for five subjects, and the program produces report cards, subject-wise and class-wise analysis, and a rank list.

The project was built using only the topics covered in the course: lists, NumPy arrays, the random module, for and while loops, if/elif/else conditions, swapping of variables, and simple functions. No external frameworks, databases or advanced libraries are used.

### Objectives

- Store names and marks of students in a clear, organised way using lists and NumPy arrays.
- Calculate total, percentage, grade and pass/fail for every student without mistakes.
- Give useful analysis: subject averages, subject toppers, class topper, pass percentage and grade distribution.
- Produce a correct rank list using bubble sort, handling students with equal totals.
- Handle wrong input safely so the program never crashes while the teacher is using it.
- Keep the code modular so each part can be understood, tested and changed on its own.

## 3. Problem Statement

Teachers calculate exam results manually or with basic spreadsheets. This is slow, repetitive and error-prone: totals can be added wrongly, a student who failed one subject may be marked as passed, and preparing a sorted rank list takes extra effort. There is a need for a simple, reliable tool that takes marks as input and automatically produces report cards, performance analysis and a rank list.

### Scope

- One class at a time, with five subjects (Maths, Science, English, Hindi, Computer). Subjects can be changed in config.py.
- Runs in the terminal; no internet, database or installation other than Python and NumPy.
- Data stays in memory while the program runs.
- Out of scope: attendance, fees, saving to files and a graphical interface (listed as future work).

## Target users

- Class teachers preparing results
- Exam / office staff
- Students checking their own grade and rank

## 4. Functional Requirements

The system has four functional modules, reached from the main menu.

| ID  | Module            | Requirement                                                                                   |
|-----|-------------------|-----------------------------------------------------------------------------------------------|
| FR1 | Record management | Add students by typing name and marks for each subject.                                       |
| FR2 | Record management | Add any number of students with random marks (for testing).                                   |
| FR3 | Record management | View all records in a table; edit a subject's marks; delete a student.                        |
| FR4 | Report cards      | Show total, percentage, grade and PASS/FAIL for all students or one student searched by name. |
| FR5 | Report cards      | Fail a student if marks in any subject are below 33, even if the percentage is high.          |
| FR6 | Analysis          | For each subject show average, highest (with topper's name) and lowest marks.                 |
| FR7 | Analysis          | Class summary: class average, topper, passed, failed, pass percentage, grade distribution.    |
| FR8 | Rank list         | Sort students by total using bubble sort; equal totals get the same rank.                     |
| FR9 | Menu              | Menu-driven workflow that keeps running until the user chooses Exit.                          |

## Input / output structure

| Input                              | Output                                           |
|------------------------------------|--------------------------------------------------|
| Menu choice (number)               | Next menu or result screen                       |
| Student name, marks for 5 subjects | Confirmation, records table                      |
| Student name (search)              | Report card of that student                      |
| Number of random students          | Records created with marks from 20 to 100        |
| —                                  | Subject analysis table, class summary, rank list |

## 5. Non-functional Requirements

| Type                         | Requirement                               | How it is met                                                                                                                                                                   |
|------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Usability                    | Easy to use without training              | Numbered menus, clear prompts, aligned tables and messages such as 'No data yet'.                                                                                               |
| Reliability / error handling | The program must not crash on wrong input | Every number is checked with <code>isdigit()</code> before <code>int()</code> ; marks outside 0–100 and empty names are asked again; options 2–4 are blocked until data exists. |
| Maintainability              | Rules easy to change                      | Subjects, pass marks and grade cutoffs are kept only in <code>config.py</code> ; each module has one job.                                                                       |
| Performance                  | Results appear instantly                  | NumPy is used for sums, means and minimums; a class of 100 students is processed in well under a second.                                                                        |
| Resource efficiency          | Low memory and no extra software          | Only two lists are stored; only NumPy is needed.                                                                                                                                |
| Testability                  | Logic can be checked automatically        | Calculation functions return values instead of printing, so <code>run_tests.py</code> can check them (35 tests).                                                                |
| Portability                  | Runs on any OS                            | Pure Python 3 terminal program; works on Windows, Linux and macOS.                                                                                                              |

## 6. System Architecture

The program follows a simple layered design. **main.py** is the presentation layer that shows the menu and calls the four functional modules. These modules use two shared helper modules: **grading.py** for all calculations and **validation.py** for checking input. Settings live in **config.py**, and the data itself (two lists) is created in `main.py` and passed to the functions that need it.

## System Architecture

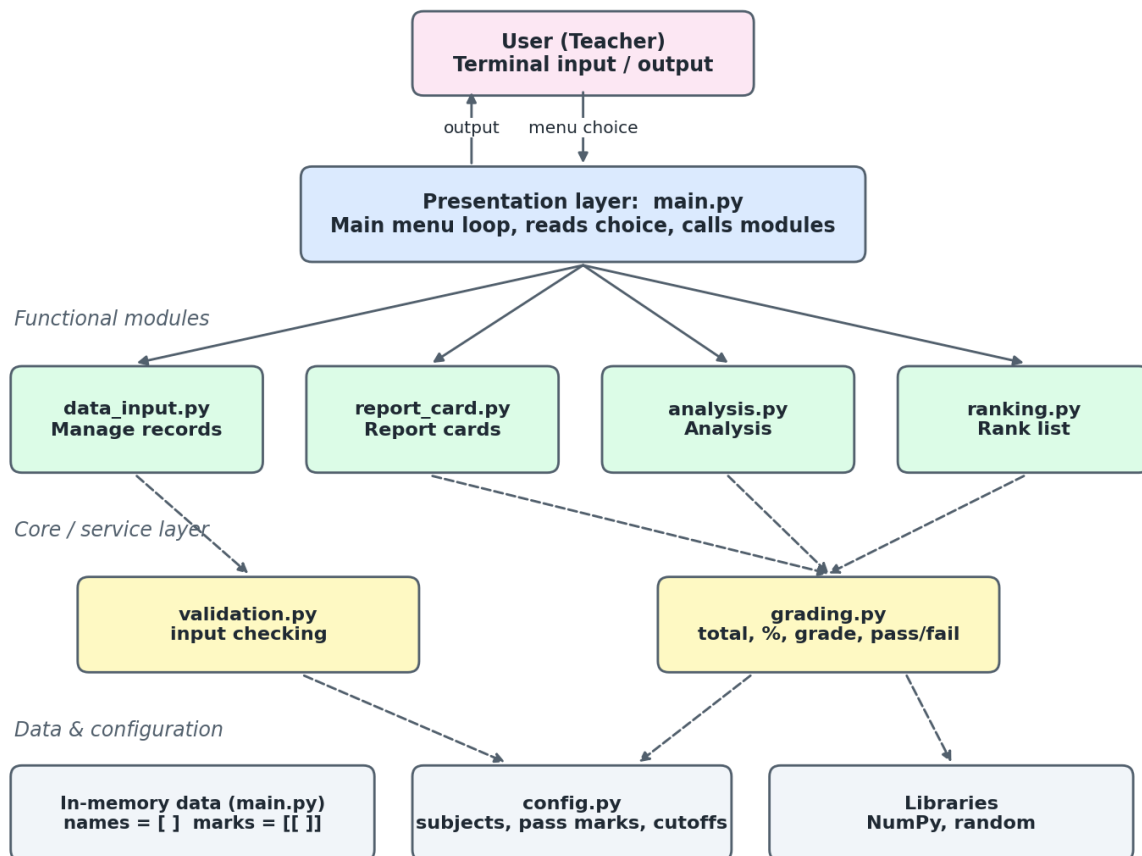


Figure 1: System architecture

## Folder structure

```

student-marks-analyser/
|-- main.py           starting point, main menu
|-- run_tests.py      validation tests
|-- requirements.txt   numpy
|-- README.md
|-- statement.md
|-- analyser/
|   |-- __init__.py
|   |-- config.py      subjects, pass marks, grade cutoffs
|   |-- validation.py  checks user input
|   |-- data_input.py  add / view / edit / delete records
|   |-- grading.py     total, percentage, grade, pass/fail
|   |-- report_card.py prints report cards
|   |-- analysis.py    subject and class analysis
|   |-- ranking.py     bubble sort rank list
|-- docs/
|   |-- images/        diagrams and screenshots
|   |-- Project_Report.pdf
  
```

## 7. Design Diagrams

### 7.1 Use case diagram

The teacher can use every feature. A student mainly views their report card and the rank list. Adding and editing marks include input validation.

## Use Case Diagram

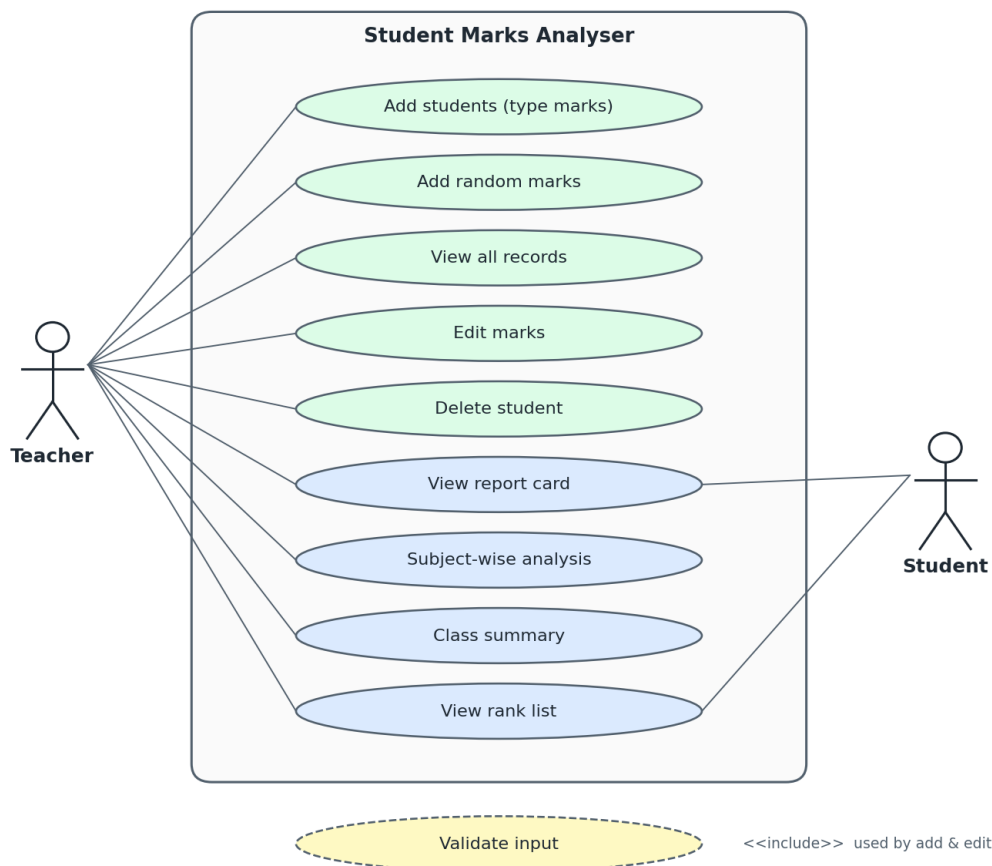


Figure 2: Use case diagram

## 7.2 Workflow diagram

The main menu runs in a while loop. Input is checked first, then the choice decides which module runs. After every task the program returns to the main menu until the user chooses Exit.

## Process Flow / Workflow

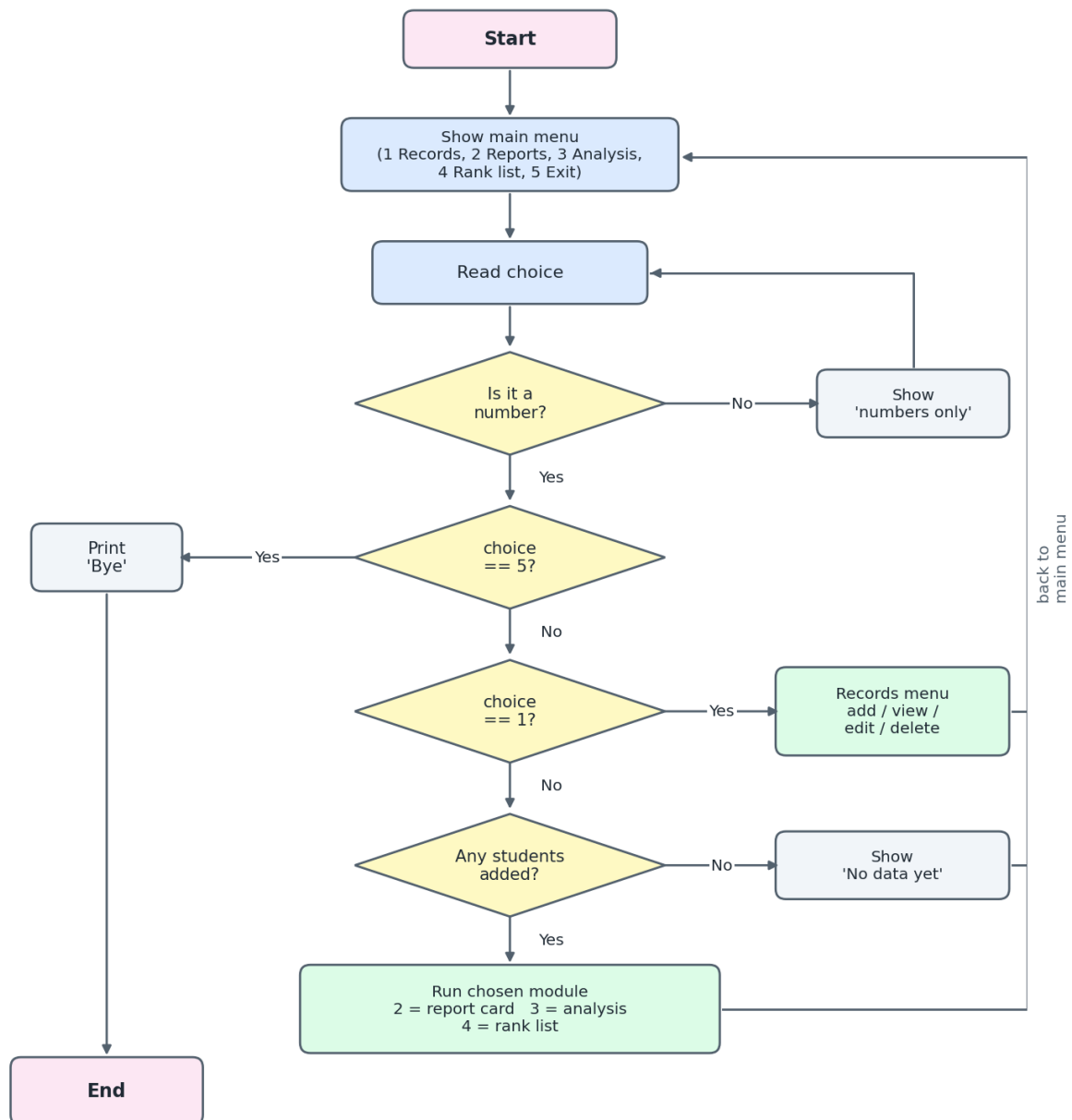


Figure 3: Process flow / workflow

### 7.3 Sequence diagram

This shows what happens when the teacher asks for one student's report card.

### Sequence Diagram: Viewing one student's report card

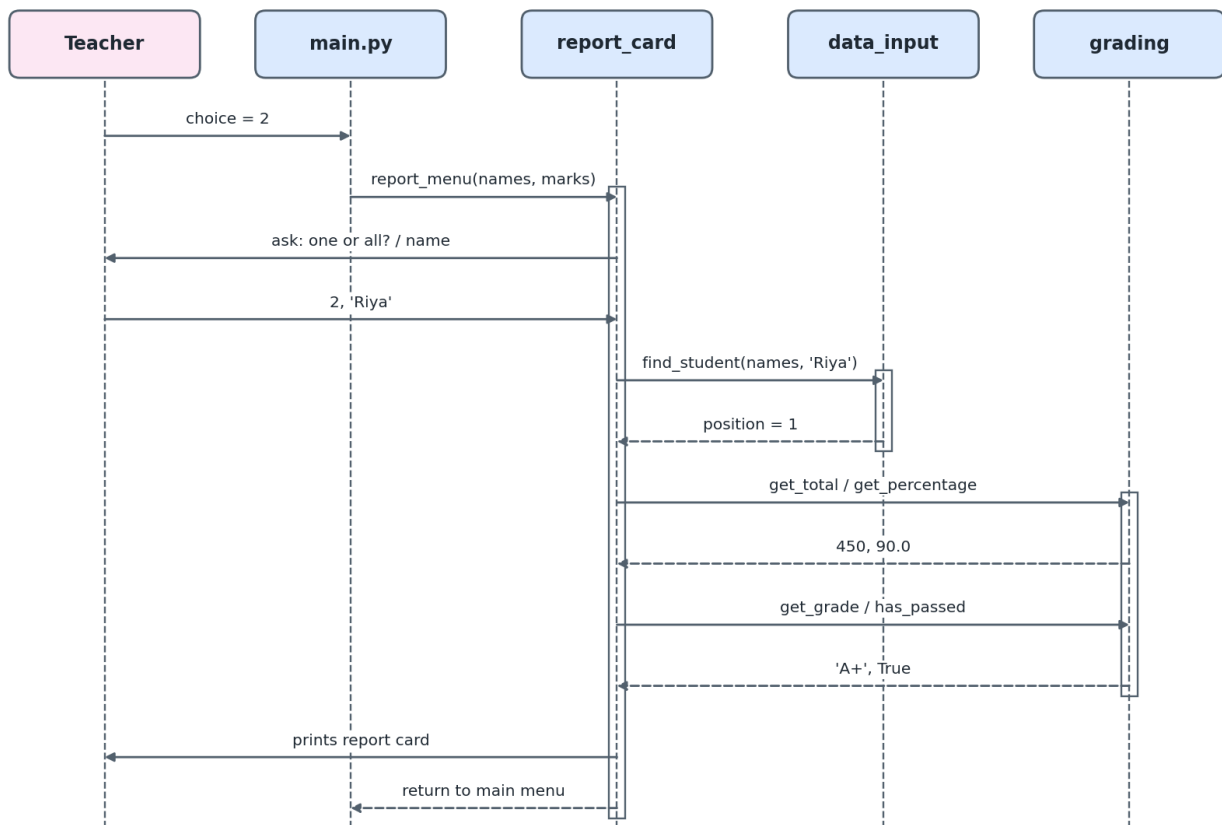
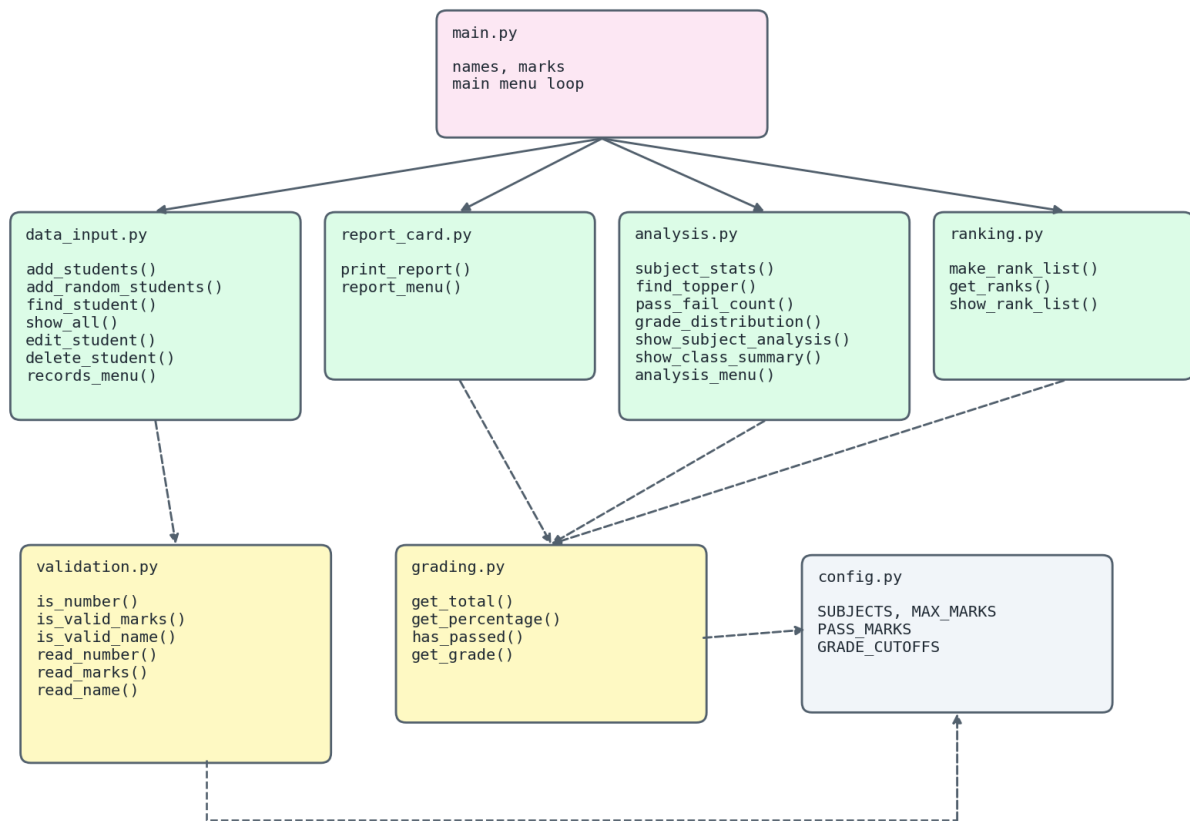


Figure 4: Sequence diagram – viewing one report card

## 7.4 Component (module) diagram

The project uses functions grouped into modules instead of classes, so a component diagram is used in place of a class diagram. It lists the functions in each file and how the files depend on each other.

## Component (Module) Diagram



solid arrow = calls, dashed arrow = uses / imports. (The project uses functions, not classes, so this module diagram replaces a class diagram.)

Figure 5: Component diagram

## 7.5 Data storage design

No database is used, so there is no ER diagram. Data is stored in memory as two parallel lists. **names[i]** and **marks[i]** always refer to the same student. When calculations are needed, marks is converted to a 2-D NumPy array, where a row is one student and a column is one subject.

## Data Storage Design (in-memory parallel lists)

|                                                               |                                                                                                                                                                                                      |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>names (list) [0] 'Amit' [1] 'Riya' [2] 'Karan' ...</pre> | <pre>marks (list of lists -&gt; np.array)       Maths Science English Hindi Computer [0] [ 50,  50,   50,   50,   50 ] [1] [ 90,  90,   90,   90,   90 ] [2] [ 70,  70,   70,   70,   70 ] ...</pre> |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

names[i] and marks[i] always belong to the same student. arr[i] = one student (row), arr[:, j] = one subject (column).

Figure 6: In-memory data design

## 8. Design Decisions and Rationale

| Decision                                                         | Reason                                                                                                                                               |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parallel lists (names, marks) instead of dictionaries or classes | These are the structures covered in the syllabus and are easy to explain. Keeping the same index for a student is enough for this problem.           |
| NumPy for calculations                                           | np.sum, np.mean and np.min make totals and averages short and correct, and arr[:, j] gives a whole subject column in one step.                       |
| Loop to find subject topper instead of np.max                    | np.max gives only the mark; a loop gives both the mark and the student's name.                                                                       |
| Bubble sort for ranking                                          | It uses swapping, which is part of the syllabus, and is fast enough for a class-size list. Names are swapped along with totals so they stay matched. |
| Fail if any subject is below 33                                  | This matches the normal school rule; a high percentage cannot hide a failed subject.                                                                 |
| isdigit() checks instead of try/except                           | Keeps to syllabus topics while still stopping crashes from letters or empty input.                                                                   |
| Settings in config.py                                            | Changing subjects or grade cutoffs needs an edit in only one place.                                                                                  |
| Functions return values; menus print                             | Calculation functions can be tested automatically, while printing stays in the menu functions.                                                       |
| Random marks option                                              | Lets the teacher (and tester) try the program with a full class in seconds.                                                                          |

## 9. Implementation Details

**Tools:** Python 3, NumPy, the random module, Git for version control. **Size:** 9 Python files, about 530 lines including comments.

### 9.1 Grading (grading.py)

The fail check runs before the grade cutoffs. Cutoffs are stored from highest to lowest, so the first one the percentage reaches is the correct grade.

```
def get_grade(student_marks):
    if not has_passed(student_marks):
        return "F"
    per = get_percentage(student_marks)
    for cut in GRADE_CUTOFFS:          # [[90,"A+"], [75,"A"], [60,"B"], ...]
        if per >= cut[0]:
            return cut[1]
    return "D"
```

### 9.2 Subject analysis with NumPy (analysis.py)

marks is turned into a 2-D array. **arr[:, j]** selects column j, which is every student's mark in one subject.

```
arr = np.array(marks)
col = arr[:, j]          # one subject, all students
highest = col[0]
top_name = names[0]
for i in range(len(col)):
    if col[i] > highest:
        highest = col[i]
        top_name = names[i]
return [round(float(np.mean(col)), 2), int(highest), top_name, int(np.min(col))]
```

### 9.3 Rank list with bubble sort (ranking.py)

```
for i in range(len(totals)):
    for j in range(len(totals) - 1 - i):
        if totals[j] < totals[j + 1]:
            totals[j], totals[j + 1] = totals[j + 1], totals[j]
            rank_names[j], rank_names[j + 1] = rank_names[j + 1], rank_names[j]
```

After sorting, **get\_ranks()** gives equal ranks to equal totals (for example 1, 2, 2, 4).

### 9.4 Input validation (validation.py)

```
def read_number(message):
    text = input(message)
    while not is_number(text):          # is_number uses text.isdigit()
        print(" Please enter a number only")
        text = input(message)
    return int(text)
```

## 9.5 Main menu (main.py)

```
while choice != 5:
    ...
    choice = validation.read_number("Enter your choice: ")
    if choice == 1:
        data_input.records_menu(names, marks)
    elif choice >= 2 and choice <= 4 and len(names) == 0:
        print("No data yet! Add students first (option 1)")
    elif choice == 2:
        report_card.report_menu(names, marks)
    ...
```

## 10. Screenshots / Results

The screenshots below use a sample class of six students.

```
Terminal - Student Marks Analyser

$ python main.py
=====
STUDENT MARKS ANALYSER
=====

===== MAIN MENU =====
1. Manage student records
2. Report cards
3. Analysis
4. Rank list
5. Exit
Enter your choice: 1

--- MANAGE RECORDS ---
1. Add students (type marks)
2. Add students with random marks
3. View all records
4. Edit marks
5. Delete a student
6. Back to main menu
Enter your choice: 1
How many students? 1
Enter name of student 1:
Name cannot be empty or only numbers
Enter name of student 1: Aarav
Marks in Maths (out of 100): abc
Please enter a number only
Marks in Maths (out of 100): 120
Marks should be between 0 and 100
Enter marks in Maths again: 88
Marks in Science (out of 100): 92
Marks in English (out of 100): 79
Marks in Hindi (out of 100): 85
Marks in Computer (out of 100): 95
1 student(s) added

--- MANAGE RECORDS ---
1. Add students (type marks)
2. Add students with random marks
3. View all records
4. Edit marks
5. Delete a student
6. Back to main menu
Enter your choice: 3

No.  Name      Maths  Science  English  Hindi  Computer
-----
1    Aarav      88      92      79      85      95

--- MANAGE RECORDS ---
1. Add students (type marks)
2. Add students with random marks
3. View all records
4. Edit marks
5. Delete a student
6. Back to main menu
Enter your choice: 6

===== MAIN MENU =====
1. Manage student records
2. Report cards
3. Analysis
4. Rank list
5. Exit
```

Figure 7: Adding a student – empty name, letters and 120 are all rejected

```
Terminal - Student Marks Analyser

$ python main.py
...
===== MAIN MENU =====
1. Manage student records
2. Report cards
3. Analysis
4. Rank list
5. Exit
Enter your choice: 3

1. Subject wise analysis
2. Class summary
Enter your choice: 1

Subject      Average   Highest   Topper      Lowest
-----
Maths        73.5      95        Diya        45
Science      69.83     97        Diya        28
English      73.83     90        Diya        48
Hindi        76.67     93        Diya        55
Computer     81.0      99        Diya        58

===== MAIN MENU =====
1. Manage student records
2. Report cards
3. Analysis
4. Rank list
5. Exit
Enter your choice: 3

1. Subject wise analysis
2. Class summary
Enter your choice: 2

Total students : 6
Class average  : 74.97 %
Class topper   : Diya ( 94.8 % )
Passed        : 5
Failed         : 1
Pass percentage : 83.33 %

Grade distribution
A+ : * 1
A  : ** 2
B  : * 1
C  : * 1
D  : 0
F  : * 1
```

Figure 8: Subject-wise analysis and class summary

```
Terminal - Student Marks Analyser

$ python main.py
...
===== MAIN MENU =====
1. Manage student records
2. Report cards
3. Analysis
4. Rank list
5. Exit
Enter your choice: 4

Rank  Name          Total
-----
1     Diya           474
2     Aarav           439
3     Sneha           423
4     Meera           372
5     Rohan           283
6     Kabir           258

===== MAIN MENU =====
1. Manage student records
2. Report cards
3. Analysis
4. Rank list
5. Exit
Enter your choice: 2

1. Report card of all students
2. Report card of one student
Enter your choice: 2
Enter student name: Kabir

=====
REPORT CARD : Kabir
=====
Maths      : 45
Science    : 28
English    : 60
Hindi      : 55
Computer   : 70
-----
Total      : 258 / 500
Percentage : 51.6 %
Grade      : F
Result     : FAIL
```

Figure 9: Rank list and report card of a student who failed Science

**Result:** Kabir scored 51.6% overall but got 28 in Science, so the program correctly gives grade F. Out of six students five passed (83.33%), and Diya is both the class topper and the topper in every subject.

## 11. Testing Approach

Two kinds of testing were done.

**1. Automated validation tests (run\_tests.py).** 35 checks using assert, run with `python run_tests.py`. Each check prints PASS, and the script stops at the first failure.

| Area       | What is checked                                                                         | Tests |
|------------|-----------------------------------------------------------------------------------------|-------|
| Grading    | Total, percentage, every grade band, fail below 33, boundary values 33 and 90%          | 10    |
| Validation | Digits, letters, empty text, negative sign, marks 0/100/101, empty and numeric names    | 10    |
| Ranking    | Descending order, correct first and last, equal ranks for ties, original list unchanged | 5     |
| Analysis   | Topper position, subject average/highest/lowest, pass-fail count, grade distribution    | 6     |
| Records    | Case-insensitive search, missing student, random students count and range               | 4     |

```

Terminal - run_tests.py

$ python run_tests.py
PASS - total of marks
PASS - percentage
PASS - grade A+
PASS - grade A
PASS - grade B
PASS - grade C
PASS - grade D
PASS - fail if one subject below 33
PASS - exactly 33 is pass
PASS - exactly 90% is A+
PASS - digits are a number
PASS - letters are not a number
PASS - empty is not a number
PASS - negative sign not allowed
PASS - marks 0 valid
PASS - marks 100 valid
PASS - marks 101 invalid
PASS - empty name invalid
PASS - number name invalid
PASS - normal name valid
PASS - rank 1 is highest total
PASS - totals sorted descending
PASS - last rank is lowest
PASS - same totals get same rank
PASS - original list not changed
PASS - topper position
PASS - subject average
PASS - subject highest
PASS - subject lowest
PASS - pass fail count
PASS - grade distribution
PASS - find student ignores case
PASS - missing student gives -1
PASS - random students added
PASS - random marks in valid range

All 35 tests passed

```

Figure 10: All 35 tests passing

**2. Manual testing.** The program was run with typed and random data to check every menu option, including wrong menu numbers, letters instead of numbers, marks above 100, empty names, searching for a student who does not exist, and choosing

options 2–4 before any data was added.

## 12. Challenges Faced

- **Keeping names matched with totals while sorting.** My first version of the rank list sorted only the totals, so the names ended up next to the wrong marks. Swapping the names in the same step fixed it.
- **Program crashing on wrong input.** Typing a letter at any prompt stopped the program because `int()` fails. I added `isdigit()` checks and while loops that keep asking until the input is valid.
- **Finding a subject topper's name.** `np.max` only returns the highest mark, not who scored it, so I used a loop to track the mark and the name together.
- **Splitting one long file into modules.** Deciding which function belongs in which file, and passing the names and marks lists between them, took some planning.
- **Students with the same total.** They were getting different ranks, so I added `get_ranks()` to give them the same rank.

## 13. Learnings and Key Takeaways

- How rows and columns of a 2-D NumPy array can be used for student-wise and subject-wise calculations.
- How bubble sort works step by step, and why swapping two values in one line is useful.
- Why input should always be checked before it is used.
- Splitting a program into small modules makes it easier to read, test and fix.
- Writing tests helped me find mistakes in boundary cases such as exactly 33 marks or exactly 90%.
- Using Git to save versions of the project as it grew.

## 14. Future Enhancements

- Save and load records from a CSV file so data is not lost when the program closes.
- Let the teacher choose the number and names of subjects when the program starts.
- Draw bar charts of subject averages and grade distribution with `matplotlib`.
- Support several classes or sections and compare them.
- A simple graphical interface (for example with Tkinter) instead of the terminal.
- Export report cards as PDF files to share with parents.

## 15. References

- Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>
- NumPy Developers, *NumPy User Guide*, <https://numpy.org/doc/stable/user/>

- Python Software Foundation, *random* — *Generate pseudo-random numbers*, <https://docs.python.org/3/library/random.html>
- Course lecture notes and lab exercises on lists, loops, conditions, swapping and NumPy.
- VITyarthi, *Build Your Own Project – General Project Instructions and Submission Guidelines*.